

Setting Up a 1-Page Website with GitHub Pages and Markdown (no HTML)

About this tutorial

This tutorial was created by Caroline T. Schroeder, licensed [CC-BY-NC 4.0](https://creativecommons.org/licenses/by-nc/4.0/). Last updated 29 August 2026.

This tutorial lives live online at <https://github.com/ctschoeder/tutorials/blob/master/GitHub-Pages-MD.md>

**I recommend you read each section carefully before doing the steps outlined in that section. This tutorial also sends you to a separate site made by GitHub to walk you through the steps. You may want to reread each of my sections after you think you've finished the section, in order to make sure you did everything correctly.

1. GitHub Account

Go to <https://github.com> and create a new account if you do not have one.

Please note: if you need to keep your identity off the internet for whatever reason, use an email address and account name that does not reveal your identity.

Go to <https://github.com> and login to your new or existing account.

2. What is GitHub?

There are a lot of videos and explainers you can watch/read on the internet. The TL/DR is that GitHub is a *commercial* site built to host code and other resources related to programming.

Sidebar: If you don't know programming languages or how to write code, **do not panic**. You will not have to use programming languages for this tutorial.

Your GitHub account is organized by **repositories**. A repository is basically a container (like a folder on your computer) for resources related to a project. Resources could be webpages, software code, documentation about your project, etc.

Files in your repositories are all saved using **version control**. This means you "commit" (rather than "save") a file and provide a description of the work you did on your file. Every time you commit your file to your repository (or "repo"), GitHub records your description and saves your file as a new **version**. GitHub does not delete your old versions. The newest version is what you see first in your repository under that file name. But you can also look at the *version history* of your file to see every version you committed to GitHub and all the descriptions. Even if you're new to GitHub, you can probably see the advantages to having all the versions of a file -- you made a mistake and need to go back to an earlier version; you cut a section out of a file and changed your mind and want it back; etc.

There is a lot more to GitHub and version control, but if you are new to GitHub, this is enough background to do the assignment.

3. Create Repository for your Webpage

Now you will set up the repo you will use to store your file for your webpage!

*If you already have a GitHub Pages site, skip down to **step 5**.*

In a new tab or window, navigate to the [GitHub Quick Start](#) page for creating your website.

- *Be sure you are logged in to your GitHub account.*
- Keep your other GitHub browser window with your account open. If you have room, position your account page and the Quick Start page side by side.
- Follow steps 1-5 on the Quick Start
- *On step 3 set the repo visibility to PUBLIC*

Yay -- you have a repo!

4. Designate Your Repo as the Source for Your Website

The feature in GitHub to host a *free* website is called GitHub Pages. In this step, you will set your new repo as the repo for your GitHub Page.

- Follow steps 6-9 on the [GitHub Quick Start](#).
- **For step 9 select "master" branch!**

Sidebar: What is a branch?

If you don't know what a **branch** is in GitHub don't worry. If you want to know: Branches are basically different lifelines of your repository. The "master" branch is the one with all the working, deployed, live versions of the your files. Other branches might be working/draft branches. For example: If you want to work on a file without making it *the file that will be used/viewed/etc. live*, you might create a "development" branch in your repo. In the "development" branch you will see copies of your files. You can work away on the files in your "development" branch and commit those changes, but those changes won't show up in the main "master" branch until you say so. (How do you say so? It's a process called a "pull request" that pulls the changes from one branch into another. But you don't need to know how to do that yet!) You can imagine that having different branches of your code might be helpful if you're building a website but you don't want your changes to go live until you're done. You can commit as many changes as you want in another branch, but those changes won't be live on your website until you *pull* the changes into the "master" branch. Here is a 15-minute YouTube video explainer.

5. Create Your Webpage

If you do not know HTML and have never used github, skip to **5c. Scenario 3**.

5a. Scenario 1: you already use GitHub and HTML

If you already have a GitHub Pages website, make a separate page on your website with a unique name (like culturalheritage.html). Follow the other instructions in section 5b for this webpage.

5b. Scenario 2: You Know HTML but you don't have a GitHub account/GitHub pages

If you don't know HTML (the core language of the web) skip to the next section titled "Using Markdown/What is Markdown?"

If you already know HTML, please create a new file in your repo.

- title it index.html
- edit index.html to make a basic HTML page about *anything* related to cultural heritage: could be your favorite heritage object or site, why you're taking the class, whatever. Please be sure:
- you use headings of some kind
- you include an image or if you don't know how to do that in GitHub then link to an image/video or other resource that pertains to what you are writing about

Note: my guess is that if you know HTML, you know how to add an image -- be sure to put the image file in your rep and then use html to add it to your index webpage

- You do not have to use your name, though if your name is in your GitHub account name the world will already know it
- choose a license for your page from [Creative Commons](#). A license will tell people who come to your webpage whether they can reuse any material there, and under what conditions.
 - Use the Chooser tool, and when you get to the "Mark Your Work" section, choose Webpage and HTML
 - Copy the HTML code and add it to the end of your WebPage

When you are done, be sure to **click on "commit changes"** to save your work!

Go to step 6.

5c. Scenario 3: You're New to GitHub and don't know HTML -- Using Markdown/What is Markdown?

For this activity, if you do not already know HTML, you will make a very basic page in what is called MarkDown language. Markdown is a much simpler list of codes to format your document.

GitHub has a convenient [list of all the formatting codes for Markdown](#))

- If you look at that list and feel overwhelmed, take a deep breath: you do NOT need to use all of those formatting codes!
- You only need to learn headings and links

Follow steps 10-11 on the [GitHub Quick Start](#) to edit your README.md file.

- that ".md" at the end of your README.md filename indicates that this is a Markdown file.
- Do not change the filename!
- Make a basic page about *anything* related to cultural heritage: could be your favorite heritage object or site, why you're taking the class, whatever. Please be sure:
- Use headings
- Link to an image/video elsewhere online, or other resource that pertains to what you are writing about
- choose a license for your page from [Creative Commons](#). A license will tell people who come to your webpage whether they can reuse any material there, and under what conditions.

Use the Chooser tool, and when you get to the "Mark Your Work" section, choose Webpage and Rich Text

When you are done, be sure to **click on "commit changes"** to save your work!

As it says in step 11, visit your website online by going to `https://username.github.io` (where "username" is *your* GitHub username)

6. Edit the title and description of your website

If you have just set up a new GitHub Pages: follow steps 1-6 in the "Changing the title and description" section on the [GitHub Quick Start](#).

Don't forget to **commit changes** to save your work!

Visit your website again -- how does it look? Feel free to edit.

7. Submit your assignment

In Canvas, under the Lab A assignment, copy and past the URL (link) for your webpage! If you are new to GitHub pages, that link should look like `https://username.github.io` (where "username" is *your* GitHub username!)

Ta da!! you're done!